Acta Informatica (2007) 44:1–21
DOI 10.1007/s00236-006-0031-y

ORIGINAL ARTICLE

Parallel *LL* parsing

Ladislav Vagner · Bořivoj Melichar

Received: 18 May 2005 / Accepted: 25 October 2006 /
Published online: 23 November 2006
© Springer-Verlag 2006

Abstract A deterministic parallel *LL* parsing algorithm is presented. The algorithm is based on a transformation from a parsing problem to parallel reduction. First, a nondeterministic version of a parallel *LL* parser is introduced. Then, it is transformed into the deterministic version — the *LLP* parser. The deterministic $LLP(q, k)$ parser uses two kinds of information to select the next operation — a lookahead string of length up to k symbols and a lookback string of length up to q symbols. Deterministic parsing is available for *LLP* grammars, a subclass of *LL* grammars. Since the presented deterministic and nondeterministic parallel parsers are both based on parallel reduction, they are suitable for most parallel architectures.

1 Introduction

Top–down parsing is a basic method that can be used to parse a string from left to right with respect to a given context-free grammar. A top–down parser is an algorithm that reads symbols from an input device, uses a pushdown store as an internal memory, and an output device to write its result to.

The algorithm performs two kinds of actions: *comparison* and *expansion*. *Comparison* consists of reading one symbol from the input string and removing one symbol from the top of the pushdown store. If these two symbols are equal, the parsing can continue, if not, the parsing is terminated and the input string is rejected. During

This research has been partly supported by The Czech Ministry of Education, Youth and Sports under research program MSM 6840770014, FRVŠ grant No. 1896/2001, and by the Czech Science Foundation as project No. 201/06/1039.

L. Vagner (✉) · B. Melichar
Department of Computer Science and Engineering, Czech Technical University in Prague,
Karlovo nám. 13, 121 35, Prague 2, Czech Republic
e-mail: xvagner@fel.cvut.cz

B. Melichar
e-mail: melichar@fel.cvut.cz

expansion, one nonterminal symbol from the top of the pushdown store is replaced by a string of terminal and nonterminal symbols. If A is the nonterminal symbol on the top of the pushdown store and $A \to \alpha$ is a rule from the context-free grammar, then A can be replaced by α in the expansion. The rule number used for the expansion is appended to the output.

The above outlined algorithm is nondeterministic. The problem is the *expansion* action. If the grammar allows rules $A \to \alpha$ and $A \to \beta$, the parser has to choose the proper rule when expanding nonterminal A. For practical purposes, however, deterministic algorithms are required. There are context-free grammars which cannot be parsed by any deterministic top–down parser. On the other hand, many practically useful context-free grammars can be. This subset of context-free grammars is named LL grammars, and their languages are named LL languages.

Parallel parsing assumes that the input string is divided into substrings which are assigned to individual processors. The shortest imaginable substring is just one symbol long. In such a case the number of processors is equal to the length of the input string. The processors must start parsing simultaneously. In particular, they cannot wait until the resulting contents of the pushdown store of the previous processors are available. For this reason, the parallel LL parsing algorithm differs from the sequential algorithm, and faces a new nondeterminism arising from the lack of information on the pushdown store.

The parallel LL parser presented here consists of two distinct steps:

1. Each processor parses its assigned substring and produces intermediate result(s). This step is, in general, nondeterministic. To reduce the nondeterminism, our parallel parser uses a lookahead string of length k or less and a lookback string of length q or less to determine the initial contents of the pushdown store.
2. The adjacent pairs of intermediate results are combined by means of parallel reduction. When the reduction terminates, one final result in the form of a left parse is obtained.

The method is not universal because only a subset of $LL(k)$ grammars can be deterministically parsed in this way. We call this class of grammars $LLP(q,k)$ grammars. The advantage of the presented parser is that it is very simple and can easily be used on any parallel computer where parallel reduction is available.

1.1 Previous work

The first parallel LL parsing algorithm that we know about was published by Skillicorn and Barnard [23]. The algorithm divides the input string into blocks which are distributed among the processors. These blocks are then parsed independently. The parsing of the blocks produces intermediate results that were named "stack configuration changes". When combined pairwise, the result describes the stack configuration change induced by a longer input string. Thus, the input string is accepted by the parser if and only if the input string produces a stack change from the starting nonterminal symbol to the empty string. The parser was designed for grammars without ε-rules, and the authors claim it is always possible to convert an $LL(k)$ grammar with ε-rules to an equivalent $LL(k+1)$ grammar without ε-rules (they cite [14], of course, the resulting grammar must contain $S \to \varepsilon$ if $\varepsilon \in L(G)$). The parser was believed to work in $O(\log(n))$ time for all LL grammars without ε-rules. The article, however, contained four major mistakes:

1. There may be more than one stack configuration change for a given input string block, thus the number of intermediate results may be very high. Even worse, when combined, the number of possible intermediate results for longer blocks may grow. There are examples where exponential growth is observed [15].
2. Due to the previous mistake, $O(\log(n))$ time cannot be achieved, because one combination of stack configuration changes no longer takes a constant time.
3. The described combination of intermediate results contained only two of the three possible cases.
4. The authors designed the parser for grammars without ε-rules only, and they recommended converting an $LL(k)$ grammar with ε-rules into an equivalent $LL(k+1)$ grammar without ε-rules (the conversion can always be made, with special treating of input string ε if $\varepsilon \in L(G)$). The conversion, however, is never needed, and introducing ε-rules only slightly complicates the design structure.

The article by Hill [10] summarizes practical experience concerning implementation of the Skillicorn's and Barnard's algorithm. The author formalizes the algorithm and specifies the "stack configuration change" combination as a parallel reduction. Next, it is noted that not all $LL(k)$ grammars without ε-rules can be used for parallel parsing. Unfortunately, the author neither specifies the subclass of LL languages where parallel parsing is available, nor gives an algorithm testing whether or not a given grammar is suitable for parallel parsing.

The dissertation thesis by Luttighuis [15] proposes a parallel LL parsing algorithm that originates from [23], fixing the above mentioned problems. The final solution is, however, very far from the original algorithm. First, the author proposes *look-back strings* to decrease the number of possible stack configuration changes induced by input string block parsing. Second, he abandons the idea of stack configuration changes and, instead, subtrees of the derivation tree are evaluated and combined. Third, the phase when the stack configuration changes are combined (called "glueing" there) is replaced by parallel bracket matching problem [19], resulting in $O(\log(n))$ time on EREW PRAM. Finally, the parser allows LL grammars with ε-rules. The thesis provides algorithms that check whether a given grammar is suitable for parallel parsing and algorithms that check whether a given subtree can be used in place of a given input string block. However, there is no algorithm evaluating all such subtrees for a given grammar. This is not generally a problem for grammars without ε-rules where the number of such subtrees is low (each subtree is described by just one root symbol), but it can be a problem for grammars with ε-rules where the "subtree" may be of forest form, with nullable nonterminals at the beginning.

Our solution is close to that of Luttighuis. There are four main differences:

1. We did not abandon the concept of stack configuration changes, and our algorithm does not use trees. In addition to easier implementation (no special data structures are needed), the parser is closer to the sequential parser in its nature.
2. We do not rely on parallel bracket matching. Although bracket matching can be used in our algorithm, parallel reduction is sufficient in many cases. There are three main reasons for using parallel reduction: first, the algorithm is simpler and does not have such a major overhead as parallel bracket matching has. Second, parallel bracket matching is a PRAM algorithm, whereas parallel reduction is available on any parallel architecture. Finally, reduction is more straightforward and is more convenient for proofs.

3. Our parallel LL parser is fully table driven, like the sequential parser. Like the generator for a sequential $LL(k)$ parser, we developed an algorithm that checks whether or not a given grammar is $LLP(q, k)$, and if it is, it produces a $PSLS$ table (a table needed by the parallel parser that corresponds to the parsing table in the sequential case). The generator algorithm is universal, for any fixed length of the lookback and lookahead string.
4. Since our parser is very close to the sequential parser, it is open for further extensions inspired by the sequential solution. When compared with sequential parsers, parallel parsers lost information about the characteristic automaton state. If this information is added to the parallel parser, the set of parsable languages might increase significantly. It should be noted that neither [15] nor our current parallel LLP parser is capable of parsing all regular languages. Such an extension could provide a way to achieve this.

A completely different parallel LL parser is proposed in [22]. The algorithm constructs a finite automaton that accepts a superset of the input language. In the first step, the parser traverses the automaton to obtain a sequence of stack operations that are induced by the input string. In the second step, this sequence is checked to find whether or not it belongs to a semi-Dyck language. Like the other parallel LL parsing algorithms, this algorithm is only suitable for a subset of LL languages.

Our previous publications concerning parallel LL parsing were in the form of extended abstracts and posters [16–18]. In [16], we outlined the basic ideas of the deterministic parallel LLP parser. In [17], we introduced an improved gluing algorithm that is, in contrast to the previous algorithm, time optimal. Finally, in [18], we sketched how the parallel LLP parser can be used to perform a formal translation.

1.2 Related work

The LR and GLR parsing algorithms are usually recognized as a better basis for parallel processing. Parallel LR parsers are (like parallel LL parsers) limited to a subset of LR languages – for example a parallel parser [21] can be constructed for the class of strong LR grammars. A different construction of a parallel LR parser is described in [12]. The idea of this parser comes from an optimized GLR parser [4]. The parallel version can be used for grammars without right and hidden left recursion.

The parallel bracket matching problem is in fact a special case of a parallel parsing algorithm, where the input language belongs to a Dyck or semi-Dyck language. Several parallel bracket matching algorithms have been published, such as [6,19]. The bracket matching problem, indeed, appears as a computation step in the described parallel LL parser.

Parallel parsing and recognition methods are often used in the field of arbitrary context-free languages. The usual approach is to parallelize CYK or Earley's [2] algorithms. Parallel parsers based on the CYK algorithm are described in [7,11,13] and parallel parsers based on Earley's algorithm are descibed in [8,20]. In [1,5,9], an overview of parallel parsing algorithms for arbitrary context-free languages is given.

The bidirectional parsing problem can be seen as a hybrid parsing method where concepts from both sequential and parallel parsing algorithms apply. The underlying theory for bidirectional parsing is studied in [24]. A linear-time algorithm for bidirectional parsing for a subset of linear grammars is given in dissertation thesis [3].

2 Notations and definitions

The set of strings over an alphabet A, including the empty string ε, is denoted by A^*. A context-free grammar is a quadruple $G = (N, T, P, S)$, where N is a finite set of nonterminal symbols, T is a finite set of terminal symbols, $T \cap N = \emptyset$, S is the starting nonterminal symbol, and P is a finite set of rules of the form $A \to \alpha$, $A \in N$, $\alpha \in (N \cup T)^*$. The symbol $\Rightarrow$ is used for the derivation relation. For any $\alpha, \beta \in (N \cup T)^*$, $\alpha \Rightarrow \beta$ if $\alpha = \gamma_1 A \gamma_2$, $\beta = \gamma_1 \gamma_0 \gamma_2$ and $A \to \gamma_0 \in P$, where $A \in N$, and $\gamma_0, \gamma_1, \gamma_2 \in (N \cup T)^*$. Symbols $\Rightarrow^k$, $\Rightarrow^+$, and $\Rightarrow^*$ are used for k-power, transitive, and transitive and reflexive closure of $\Rightarrow$, respectively. The symbol $\Rightarrow_{lm}$ is reserved for the leftmost derivation, e.g. $\gamma_1 A \gamma_2 \Rightarrow_{lm} \gamma_1 \gamma_0 \gamma_2$ if $\gamma_1 \in T^*$. A sentential form α is a string which can be derived from S, that is, $S \Rightarrow^* \alpha$. The sentential form α such that $S \Rightarrow^*_{lm} \alpha$ is called the leftmost sentential form. The formal language generated by the grammar $G = (N, T, P, S)$ is the set of strings $L(G) = \{w : S \Rightarrow^* w, w \in T^*\}$. Two grammars G_1 and G_2 are equivalent if $L(G_1) = L(G_2)$. The set $N_\varepsilon = \{A : A \Rightarrow^* \varepsilon, A \in N\}$ is called a set of *nullable* nonterminal symbols.

The set of all terminal strings of length up to k symbols is denoted by T^{*k} where $k > 0$. Formally, $T^{*k} = \{x : x \in T^*, |x| \leq k\}$, where the length of string $x \in T^*$ is denoted by $|x|$. We define the sets $\text{FIRST}_k(\alpha)$ and $\text{FOLLOW}_k(A)$ with respect to a given context-free grammar G, as follows:

$$\begin{aligned}
\text{FIRST}_k(\alpha) &= \left\{x : x \in T^* : \alpha \Rightarrow^* x\beta \text{ and } |x| = k\right\} \cup \\
&\quad \left\{x : x \in T^* : \alpha \Rightarrow^* x \text{ and } |x| \leq k\right\} \\
\text{FOLLOW}_k(A) &= \left\{x : x \in T^* : S \Rightarrow^* \alpha A \beta \text{ and } x \in \text{FIRST}_k(\beta)\right\}
\end{aligned}$$

The configuration of an LL parser is a triple (w, α, π), where $w \in T^*$ is the not yet processed part of the input string, α is the contents of the pushdown store (the topmost symbol is on the left), and π a prefix of a left parse.

3 Parallel *LL* parsing

The basic idea of the parser is similar to the idea of the parallel LL parser proposed in [10,23]. However, we define the parser in a more detailed manner and we discuss some problems and limitations not mentioned in [23]. Moreover, we show that a parallel LL parser can be constructed even if there are ε-rules in the grammar.

3.1 The parallel parser

The main difference between sequential and parallel parsers is the access to the input string. A sequential parser reads the input from left to right, one symbol at a time while a parallel LL parser partitions the input string and assigns substrings to the individual processors. Without loss of generality, we will further assume that the string is partitioned into substrings just one symbol long and that the number of processors is equal to the number of symbols in the input string. If there were fewer processors than the length of the input string, the string would be partitioned into longer substrings and each processor would be assigned a string longer than one input symbol. In such a case, the processor would sequentially simulate all the processors which would

be in the corresponding subtree if the finest granularity partitioning were used. The algorithm, however, remains the same.

The parallel LL parser recognizes two main phases: *parsing* and *gluing*. Parsing takes place first and can be understood as a preparation for parallel reduction. The second phase is called gluing. It connects the intermediate results by means of parallel reduction. After this step, the processor in the root holds the left parse of the input string or an error signaling if the input string was not accepted. These two phases are described in detail below.

Throughout the paper, we use the following notation: processors are denoted P_{ij}, where i means level in the reduction tree ($i = 1$ for leaves) and j is the processor index in its level ($j = 1$ for the leftmost one). We use the terms left-hand and right-hand child in the usual way: processors $P_{i-1,\frac{j}{2}-1}$ and $P_{i-1,\frac{j}{2}}$ are left-hand and right-hand children of processor P_{ij}, respectively.

If the input string is $x \in T^*, x = a_1a_2a_3 \ldots a_n$, then processors $P_{11}, P_{12}, \ldots, P_{1n}$ will be assigned input symbols $a_1, a_2, \ldots, a_n$, respectively. If more than one process runs on processor P_{1i}, then the processes are denoted $Q_1(a_i), Q_2(a_i), \ldots, Q_j(a_i)$.

3.1.1 Parsing phase

Parallel parsing starts in the leaf processors which perform the LL parsing itself. Each leaf processor, of course, needs to know the initial contents of its pushdown store in order to start the parsing. At this moment, we will assume that each processor P_{1i} can somehow obtain its initial pushdown store contents α_i. Note that there might exist more than one such α_i, in which case the processor spawns several processes $Q(a_i)$. Having this, the parsing of symbol a_i can be accomplished—each process $Q(a_i)$ performs a sequence of expansions and finishes with one comparison of the symbol a_i:

$$(a_i, \alpha_i, \varepsilon) \vdash^* (\varepsilon, \omega_i, \pi_i)$$

Each leaf processor provides intermediate results which can be of two kinds:

- The processor successfully parsed the input symbol a_i. In this case, the results are triplets $(\alpha_i, \omega_i, \pi_i)$ containing the initial pushdown store contents, the final pushdown store contents and a portion of the left parse. Note that there may be several different triplets that can be used to parse a_i.
- The parsing fails. In such a case, a special triplet (**error**, **error**, **error**) is generated, and this triplet acts as an error signaling.

3.1.2 Gluing phase

The second phase is parallel reduction which glues the triplets generated by the leaf processors. The task of gluing of processes is to put the partial parses together and so to obtain a complete left parse of the input string. There is information, for each partial parse, on the contents of the pushdown store at the beginning and at the end of the leaf parsing process.

Recall that triplet $(\alpha_i, \omega_i, \pi_i)$ means that an LL parser with initial pushdown store contents equal to α_i and input string a_i will read the input, its final pushdown store contents will be equal to ω_i and a portion of the left parse will be π_i. The reduction is constructed such that this property is extended to strings longer than one symbol. Let

P_{ji} be an arbitrary processor and let $(\alpha_{ji}, \omega_{ji}, \pi_{ji})$ be its intermediate result. Further, let $a_k, a_{k+1}, \ldots, a_{k+l}$ be the input string symbols assigned to leaf processors in the subtree rooted in P_{ji}, $k = 2^{j-1}(i-1)+1$, $l = 2^{j-1}$. Then the following holds:

$$(a_k a_{k+1} \cdots a_{k+l}, \alpha_{ji}, \varepsilon) \vdash^* (\varepsilon, \omega_{ji}, \pi_{ji}) \tag{1}$$

Definition 1 *Let triplets $(\alpha_l, \omega_l, \pi_l)$ and $(\alpha_r, \omega_r, \pi_r)$ be intermediate results of a left-hand side processor and a right-hand processor, respectively. These results may be glued after comparing of ω_l and α_r only in the following cases:*

1. *$\omega_l = \alpha_r$, the resulting triplet will be $(\alpha_l, \omega_r, \pi_l\pi_r)$.*
2. *ω_l is a prefix of α_r, e.g., $\alpha_r = \omega_l\beta$ and $\beta \neq \varepsilon$. The resulting triplet will be $(\alpha_l\beta, \omega_r, \pi_l\pi_r)$.*
3. *α_r is a prefix of ω_l, e.g., $\omega_l = \alpha_r\beta$ and $\beta \neq \varepsilon$. The resulting triplet will be $(\alpha_l, \omega_r\beta, \pi_l\pi_r)$.*
4. *Neither α_r is a prefix of ω_l nor ω_l is a prefix of α_r, e.g., $\alpha_r = \delta\sigma_1$, $\omega_l = \delta\sigma_2$, $\sigma_1 \neq \sigma_2$, $|\sigma_1| \geq 1$, $|\sigma_2| \geq 1$. In such a case gluing will generate a special error triplet* (**error**, **error**, **error**) *as a result.*
5. *The left or right intermediate result is a special error triplet* (**error**, **error**, **error**). *In such a case the error signaling is simply propagated up* (*an error triplet acts as an aggressive element for gluing*).

In the further text, the gluing operation will be denoted as an infix operator **glue**.

Theorem 2 *The gluing of intermediate results preserves* (1)*, i.e. if triplet (α, ω, π) is the intermediate result of gluing in a subtree node whose leaves are labeled $a_1 a_2 \cdots a_n = x$, then the sequential LL parsing algorithm can perform $(x, \alpha, \varepsilon) \vdash^* (\varepsilon, \omega, \pi)$.*

Proof By induction according to the length of the input string.

1. Basis: The relation (1) holds for all input strings one symbol long: let (α, ω, π) be a triplet produced by a leaf processor after input string x of the length of one symbol has been read. Then $(x, \alpha, \varepsilon) \vdash^* (\varepsilon, \omega, \pi)$ follows from the definition of leaf processor parsing.
2. Induction: Let (1) hold for all input strings of length less or equal to some fixed k, $k > 1$. Let x be a string of length $k+1$, $x = x_l x_r$, where the length of both x_l and x_r is less or equal to k. Further, let $(x_l, \alpha_l, \varepsilon) \vdash^* (\varepsilon, \omega_l, \pi_l)$ and $(x_r, \alpha_r, \varepsilon) \vdash^* (\varepsilon, \omega_r, \pi_r)$ and let triplet (α, ω, π) be the result of gluing, i.e. $(\alpha, \omega, \pi) = (\alpha_l, \omega_l, \pi_l)$ **glue** $(\alpha_r, \omega_r, \pi_r)$. We will show that the previous implies that $(x, \alpha, \varepsilon) \vdash^* (\varepsilon, \omega, \pi)$.
   1. If $\alpha_r = \omega_l$, then:
      $(x, \alpha, \varepsilon) = (x_l x_r, \alpha_l, \varepsilon) \vdash^*$
      $(x_r, \omega_l, \pi_l) = (x_r, \alpha_r, \pi_l) \vdash^*$
      $(\varepsilon, \omega_r, \pi_l\pi_r) = (\varepsilon, \omega, \pi)$.
   2. If α_r is a prefix of ω_l, e.g. $\omega_l = \alpha_r\gamma$, then:
      $(x, \alpha, \varepsilon) = (x_l x_r, \alpha_l, \varepsilon) \vdash^*$
      $(x_r, \omega_l, \pi_l) = (x_r, \alpha_r\gamma, \pi_l) \vdash^*$
      $(\varepsilon, \omega_r\gamma, \pi_l\pi_r) = (\varepsilon, \omega, \pi)$.
   3. If ω_l is a prefix of α_r, e.g. $\alpha_r = \omega_l\gamma$, then:
      $(x, \alpha, \varepsilon) = (x_l x_r, \alpha_l\gamma, \varepsilon) \vdash^*$
      $(x_r, \omega_l\gamma, \pi_l) = (x_r, \alpha_r, \pi_l) \vdash^*$
      $(\varepsilon, \omega_r, \pi_l\pi_r) = (\varepsilon, \omega, \pi)$.

The previous three situations describe all successful gluings, thus the theorem is proved. □

Clearly, Theorem 2 is not an equivalence. Failing to glue an intermediate results pair does not imply that there does not exist another pair whose gluing can succeed. Thus, we can claim a somewhat weaker theorem:

Theorem 3 *Let $x = x_1x_2$ be an input string and let K_1 and K_2 be sets of all triplets such that $K_1 = \{(\alpha,\omega,\pi) : (x_1,\alpha,\varepsilon) \vdash^* (\varepsilon,\omega,\pi)\}$ and $K_2 = \{(\alpha,\omega,\pi) : (x_2,\alpha,\varepsilon) \vdash^* (\varepsilon,\omega,\pi)\}$. If for each pair $\tau_1 \in K_1$ and $\tau_2 \in K_2$ the gluing of intermediate results τ_1* **glue** *τ_2 provides* (**error**, **error**, **error**) *as a result, then the sequential LL parser performs transition $(x,\gamma,\varepsilon) \vdash^*$ error for any initial pushdown store contents $\gamma \in (N \cup T)^*$.*

Proof By contradiction. Let there exist a pushdown store content α such that $(x,\alpha,\varepsilon) \vdash^* (\varepsilon,\omega,\pi)$ and let the gluing fail for all pairs τ_1, τ_2, $\tau_1 \in K_1$, $\tau_2 \in K_2$.

Since the LL parser can perform the transition, there exists $\alpha_1 \in (N \cup T)^*$, $\alpha = \alpha_1\beta$, such that $\alpha_1 \Rightarrow_{lm} x_1\gamma$, where $\beta,\gamma \in (N \cup T)^*$. It follows that $(x_1,\alpha_1,\varepsilon) \vdash^* (\varepsilon,\gamma,\pi_1)$ and thus triplet $\tau_l = (\alpha_1,\gamma,\pi_1) \in K_1$. Let us proceed with the LL automaton transition: $(x,\alpha,\varepsilon) = (x_1x_2,\alpha_1\beta,\varepsilon) \vdash^* (x_2,\gamma\beta,\pi_1) \vdash^* (\varepsilon,\omega,\pi_1\pi_2) = (\varepsilon,\omega,\pi)$. It is now obvious that $(x_2,\gamma\beta,\varepsilon) \vdash^* (\varepsilon,\omega,\pi_2)$, thus $\tau_r = (\gamma\beta,\omega,\pi_2) \in K_2$. But triplets τ_l and τ_r can be glued, which is in contradiction with the conditions. □

Theorem 2 guarantees that gluing will not introduce new input strings that the sequential LL parser does not accept, and Theorem 3 guarantees that gluing will not throw away a potentially acceptable input string.

Theorem 4 *Gluing is an associative operation* (*thus it can be used in parallel reduction*).

Proof Let there be configuration triplets $(\alpha_1,\omega_1,\pi_1)$, $(\alpha_2,\omega_2,\pi_2)$, and $(\alpha_3,\omega_3,\pi_3)$. We have to prove that gluing is an associative operation, thus:

$$\Big((\alpha_1,\omega_1,\pi_1)\ \textbf{glue}\ (\alpha_2,\omega_2,\pi_2)\Big)\ \textbf{glue}\ (\alpha_3,\omega_3,\pi_3)$$
$$= (\alpha_1,\omega_1,\pi_1)\ \textbf{glue}\ \Big((\alpha_2,\omega_2,\pi_2)\ \textbf{glue}\ (\alpha_3,\omega_3,\pi_3)\Big)$$

holds for all intermediate results. The proof simply examines all possible cases. We do not include the proof itself in the text, as it is rather long and does not contain any inventive ideas. □

The gluing stops in the root processor. The processor will produce configurations $(\alpha_f,\omega_f,\pi_f)$. Thanks to Theorems 2 and 3, we can state that the input string is accepted if and only if there is a configuration such that α_f is equal to the starting nonterminal symbol and ω_f is the empty string. In such a case, π_f contains the left parse of the input string. The input string is rejected in all other cases.

3.2 Nondeterministic parallel LL parsing

We defined the parallel LL parser in Sect. 3.1. Now, we will demonstrate how such a parser works. Throughout this paper, we will use the working example context-free $LL(1)$ grammar $G = (N,T,P,E)$, where the set of nonterminals $N = \{E,E',T\}$, the set of terminals $T = \{a,+,[,]\}$, and P consists of the following rules:

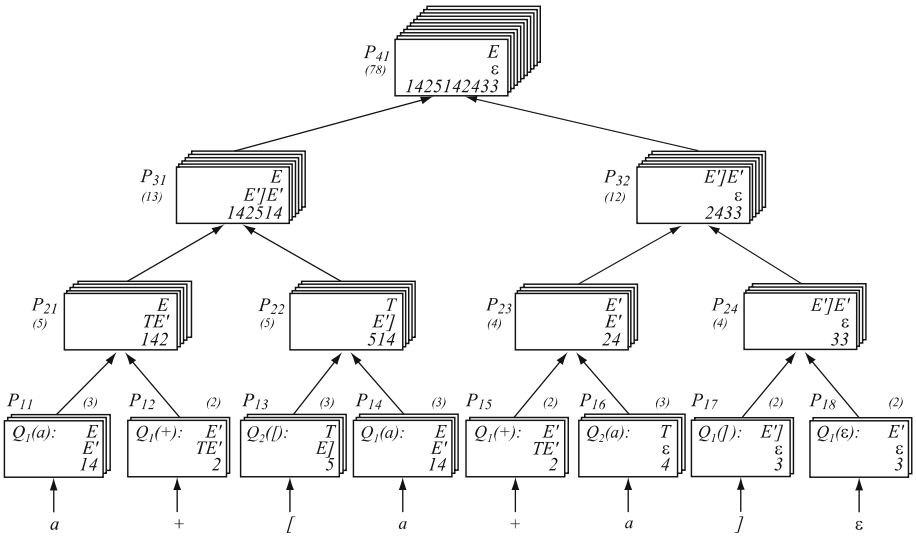

Fig. 1 Nondeterministic parallel parsing of input string $a + [a + a]$

$$
\begin{array}{llllll}
(1) & E \to TE' & (2) & E' \to +TE' & (3) & E' \to \varepsilon \\
(4) & T \to a & (5) & T \to [E] & &
\end{array}
$$

The example grammar is a simplification of a grammar that generates arithmetic expressions. We have chosen this grammar since it is useful in practice, contains ε-rules, and is not a linear grammar. The parallel parser can be constructed for the complete arithmetic expression grammar (which contains operators with different priorities), but we will use this simplification since it keeps the examples shorter.

The *LL* parsing table for this grammar is:

	a	$+$	$[$	$]$	ε
E	(1)		(1)		
E'		(2)		(3)	(3)
T	(4)		(5)		

For the parallel parsing of input string $a + [a + a]$, we use a parallel system with the processor network depicted in Fig. 1. Note that the processor network includes one extra processor that is to parse the ε-suffix of the input string. The definition of parsing (Sect. 3.1.1) assumes that each leaf processor performs a sequence of expansions followed by a comparison. It would not be possible to build a parser on this assumption if the last operation in the parsing were an expansion. This is the case if the grammar allows leftmost sentential forms with nullable nonterminals at the end. For the case of nondeterministic parsing, this limitation can be solved by adding one extra processor at the end. The processor either does nothing (i.e. produces triplet $(\varepsilon, \varepsilon, \varepsilon)$) if there are no nullable nonterminals to be expanded, or performs the required expansions by ε-rules, providing the appropriate triplet.

The ideal parsing algorithm is based on knowledge of the contents of the pushdown store. However, in the reality, this information is not available because the contents of the pushdown store are known after having parsed the preceding part of the input string. Therefore, the leaf processors must guess the contents of the pushdown store

$$
\begin{array}{llllll}
Q_1(a): & (a, E, \varepsilon) & \vdash (a, TE', 1) & \vdash (a, aE', 14) & \vdash (\varepsilon, E', 14) \\
Q_2(a): & (a, T, \varepsilon) & \vdash (a, a, 4) & \vdash (\varepsilon, \varepsilon, 4) & \\
Q_3(a): & (a, a, \varepsilon) & \vdash (\varepsilon, \varepsilon, \varepsilon) & & \\
Q_1([): & ([, E, \varepsilon) & \vdash ([, TE', 1) & \vdash ([, [E]E', 15) & \vdash (\varepsilon, E]E', 15) \\
Q_2([): & ([, T, \varepsilon) & \vdash ([, [, 5) & \vdash (\varepsilon, \varepsilon, 5) & \\
Q_3([): & ([, [, \varepsilon) & \vdash (\varepsilon, \varepsilon, \varepsilon) & & \\
Q_1(]): & (], E'], \varepsilon) & \vdash (],], 3) & \vdash (\varepsilon, \varepsilon, 3) & \\
Q_2(]): & (],], \varepsilon) & \vdash (\varepsilon, \varepsilon, \varepsilon) & & \\
Q_1(+): & (+, E', \varepsilon) & \vdash (+, +TE', 2) & \vdash (\varepsilon, TE', 2) & \\
Q_2(+): & (+, +, \varepsilon) & \vdash (\varepsilon, \varepsilon, \varepsilon) & & \\
Q_1(\varepsilon): & (\varepsilon, E', \varepsilon) & \vdash (\varepsilon, \varepsilon, \varepsilon) & & \\
Q_2(\varepsilon): & (\varepsilon, \varepsilon, \varepsilon) & \vdash^0 (\varepsilon, \varepsilon, \varepsilon) & &
\end{array}
$$

Fig. 2 Possible leaf proccesses for the example grammar

using the parsing table. In general, the initial pushdown store contents may have one of the following three forms:

- First, the pushdown store contents may be equal to the input symbol being parsed. In such a case, the parser simply performs a comparison.
- Second, several other choices can be obtained from the parsing table. Depending on the number of non-error entries in the column for a given lookahead string, there may exist several different contents of the pushdown store and therefore there are several ways to start the parsing of the symbol. The pushdown store contents can be any of the nonterminals that correspond to the rows of the parsing table where non-error entries were found.
- Finally, either of the above obtained initial pushdown store contents may provide several other choices when prepended by a sequence of nullable nonterminals. The number of such choices can even be infinite. The case is, however, artificial and can be disregarded for practical grammars. Suppose a nullable nonterminal X and initial pushdown store contents $XXX \cdots X\gamma$, where the X can occur an arbitrary number of times. These pushdown store contents correspond to some portion of a leftmost sentential form. Since X is nullable, $X \Rightarrow^* \varepsilon$. Next, the grammar is an LL grammar, thus there cannot be $X \Rightarrow^* u, u \neq \varepsilon$. If such u existed, this would cause a FIRST-FOLLOW conflict. Thus we can state that the only string that can be derived from X is the empty string. But then X can be left out from the grammar without changing the language. The conclusion is that the number of initial pushdown store contents is finite for practical grammars, since a nullable nonterminal may appear at most a fixed number of times in the prependend string (especially, at most once for $LL(1)$ grammars). The same reasoning applies if there is more than one nullable nonterminal in the grammar – each of them may occur at most a fixed number of times.

Having established all possible initial pushdown store contents, we will spawn a parsing process for each one, all in parallel. Figure 2 lists the processes for all possible initial pushdown store contents for our example grammar. Note that we only list the processes with an initial pushdown store that correspond to some portion of the leftmost sentential form. For instance, initial pushdown store contents $E'a$ is not listed since it may not appear in any leftmost sentential form.

The correct parsing of input string $a + [a + a]$ is the sequence of these eight leaf parsing processes: $Q_1(a), Q_1(+), Q_2([), Q_1(a), Q_2(+), Q_2(a), Q_1(]), Q_1(\varepsilon)$ producing the left parse 14 2 5 14 2 4 3 3.

As we have seen, the parallel leaf parsing of input symbols involves nondeterminism. However, gluing the previous results in subsequent steps also leads to nondeterminism. Let us assume the example input string $a + [a + a]$, and let us discuss gluing in processor P_{23}. According to Fig. 2, there are two possible intermediate results for the left child (symbol $+$) and three possible intermediate results for the right child (symbol a). In our case, there are four allowed gluing results from the maximum of six possible, namely $Q_1(+)Q_2(a)$, $Q_2(+)Q_1(a)$, $Q_2(+)Q_2(a)$, and $Q_2(+)Q_3(a)$.

The above example demonstrates that the number of intermediate results can increase and become impractically high (in the worst case, the number can grow exponentially [15]). On the other hand, this situation may be avoided if each leaf processor can positively evaluate its initial pushdown store contents.

The number of possible intermediate results is also depicted in Fig. 1 – the actual number is written in parenthesis close to each processor.

3.3 Deterministic parallel LL parsing

To reduce nondeterminism, leaf processors are given information on whether their input symbol is the first, the last, or a middle one. This can be done by adding left and right markers to input string w yielding input string $\vdash w \dashv$. If these markers are not in T, the parser will have information on the position of the parsed symbol. Thanks to these markers, we can leave out the last processor parsing ε-suffix of the input string, since the markers guarantee that the last operation in the parsing will be the comparison of the right marker. However, this removes neither leaf parsing nondeterminism, nor gluing nondeterminism.

To avoid the latter two types of nondeterminism, leaf processors are given information on their lookback strings. This additional information (together with the lookahead string), in general, restricts the possible initial pushdown store contents, and may even lead to the ideal situation, where at most one initial pushdown store contents is available for an arbitrary lookahead and lookback string pair. Grammars where this condition holds for some fixed lookahead string length up to k symbols and lookback string up to q symbols can be deterministically parsed in parallel; we have named them $LLP(q, k)$ grammars.

To cope with $LLP(q, k)$ grammars, we need to determine pairs (u, v), $u \in T^{*q}$, $v \in T^{*k}$, for which entries in the table of an $LLP(q, k)$ grammar will be nonerror entries. These pairs will be called *admissible pairs*. Later in the text, an algorithm evaluating admissible pairs will be introduced.

3.4 $LLP(q, k)$ grammars

The algorithms presented in this section solve the two crucial questions concerning $LLP(q, k)$ grammars:

1. How to test whether or not a given context-free grammar is an $LLP(q, k)$ grammar?
2. How to evaluate the initial pushdown store contents for a given lookahead and lookback string pair?

Since lookback strings are often evaluated and tested, we will introduce two new functions LAST and BEFORE. These functions are used to evaluate the terminal symbol strings that may appear at the end of derivation from α and before nonterminal symbol X, respectively.

Definition 5 *Let $G = (N, T, P, S)$ be a context-free grammar, α be a string of symbols $\alpha \in (N \cup T)^*$, A be a nonterminal symbol $A \in N$, and q be a integer, $q \geq 1$. The functions are defined as follows*:

$$\begin{aligned}\mathrm{LAST}_q^G(\alpha) &= \left\{x : x \in T^* : \alpha \Rightarrow^* \beta x \text{ and } |x| = q\right\} \cup \\ &\quad \left\{x : x \in T^* : \alpha \Rightarrow^* x \text{ and } |x| \leq q\right\}, \\ \mathrm{BEFORE}_q^G(A) &= \left\{x : x \in T^* : S \Rightarrow^* \alpha A \beta \text{ and } x \in LAST_q(\alpha)\right\}.\end{aligned}$$

Note that we will omit indices q and G where the length and grammar is clear from the text context.

These functions are similar to FIRST and FOLLOW, moreover, we can use the already developed algorithms for FIRST and FOLLOW to compute the values of LAST and BEFORE. If G is a context-free grammar, the following holds:

$$\begin{aligned}\mathrm{LAST}_q^G(\alpha) &= \left(\mathrm{FIRST}_q^{G^R}(\alpha^R)\right)^R, \\ \mathrm{BEFORE}_q^G(X) &= \left(\mathrm{FOLLOW}_q^{G^R}(X)\right)^R,\end{aligned}$$

G^R is a reversed grammar (i.e. the right-hand sides of all rules in G^R are reversed) and α^R is a string reversal.

Next, we will define function PSLS (Prefix of Suffix of Leftmost Sentential form) for pairs of strings (x, y). Given a pair of strings (x, y), the function specifies the set of all possible prefixes α of the pushdown store contents that may appear on the pushdown store of a standard LL parser when the following conditions hold:

- the string xy appears in the input string as a substring,
- the parser parsed the input string and the input head moved just after the last symbol of x,
- the last operation performed by the parser was the comparison of the last symbol of x, and no further expansion was performed.

The strings α included in the set are restricted in length. The strings must be long enough to allow derivation of the first symbol from y, on the other hand, only the shortest such prefixes are included. That is, if α and β both fulfill the above critera and α is a prefix of β, then only α is included in $PSLS(x, y)$.

Definition 6 *Let $G = (N, T, P, S)$ be a context-free grammar. The function $PSLS(x, y)$ for a pair of strings $x, y \in T^*$ is defined as follows:*

$$\begin{aligned}\mathrm{PSLS}(x, y) = \Big\{&\alpha : \exists S \Rightarrow_{lm}^* wuA\beta \Rightarrow wxB\gamma \Rightarrow^* wxy\delta, \\ &w, u \in T^*, A, B \in N, \alpha, \beta, \gamma, \delta \in (N \cup T)^*, u \neq x, \\ &\alpha \text{ is the shortest prefix of } B\gamma \text{ such that } y \in \mathrm{FIRST}_1(y) \subseteq \mathrm{FIRST}_1(\alpha)\Big\} \\ \cup \Big\{&a : \exists S \Rightarrow^* wuA\beta \Rightarrow wxa\gamma \Rightarrow^* wxy\delta, \\ &a = \mathrm{FIRST}_1(y), w, u \in T^*, \beta, \gamma, \delta \in (N \cup T)^*, u \neq x\Big\}\end{aligned}$$

The PSLS function gives us an answer to both the questions from the beginning of this section. The only remaining problem is to how to evaluate its values. We present an algorithm based on a collection of sets of $LLP(q, k)$ items.

Definition 7 *Let $G = (N, T, P, S)$ be a context-free grammar. An $LLP(q, k)$ item for G is a quadruple having the form $\left[X \to \alpha\bullet\beta, u, v, \gamma\right]$, where $X \to \alpha\beta$ is a rule in P, $u \in T^{q*}$ is a string of terminals that may appear before the dot sign in a leftmost derivation from the starting nonterminal symbol S, $v \in T^{k*}$ is a string from* $\mathrm{FIRST}_k(\beta\mathrm{FOLLOW}_k(X))$, *$\gamma \in (T \cup N)^*$ is a prefix of a suffix of the leftmost sentential form that can generate string v. Formally, let $S \Rightarrow^*_{lm} \omega_1 X\omega_2 \Rightarrow_{lm} \omega_1\alpha\beta\omega_2$, $\omega_1 \in T^*$, $\alpha, \beta, \omega_2 \in (N \cup T)^*$, then*

- *u is a suffix of the terminal string that can be derived from sentential form $\omega_1\alpha$, i.e. $\omega_1\alpha \Rightarrow^* \omega_3 u$, $\omega_3 \in (N \cup T)^*$,*
- *v is a prefix of the terminal string that can be derived from sentential form $\beta\omega_2$, i.e. $\beta\omega_2 \Rightarrow^* v\omega_4$, $\omega_4 \in (N \cup T)^*$, and*
- *γ is the shortest prefix of $\beta\omega_2$ having sufficient length such that $\gamma \Rightarrow^*_{lm} v\omega_5$, $\omega_5 \in (N \cup T)^*$.*

Algorithm 8 Construction of a collection of sets of $LLP(q, k)$ items.
Input A context-free grammar $G = (N, T, P, S)$.
Output A collection C of sets of $LLP(q, k)$ items for G.

1. The grammar is augmented in the following way:

$$G' = (N \cup \{S'\}, T \cup \{\vdash, \dashv\}, P \cup \{S' \to \vdash S \dashv\}, S'),$$

   where S' is a new nonterminal symbol and $\vdash, \dashv$ are new terminal symbols.
2. The initial set of $LLP(q, k)$ items is constructed as follows:
   (a) $D_0 := \left\{\left[S' \to \vdash S \dashv \bullet, u, \varepsilon, \varepsilon\right]\right\}$, $u = \mathrm{LAST}_q(\vdash S \dashv)$.
   (b) $C := \{D_0\}$.
3. If a set of $LLP(q, k)$ items has been constructed, then a new set D_j is constructed for each symbol $X \in (N \cup T)$ standing just before the dot in D_i. The set D_j is constructed as follows:
   (a) $D_j := \left\{\left[Y \to \alpha\bullet X\beta, u_j, v_j, \gamma\right]\right.$, where $[Y \to \alpha X\bullet\beta, u_i, v_i, \delta] \in D_i$, $u_j \in \mathrm{LAST}_q(\mathrm{BEFORE}_q(Y)\alpha)$, $v_j \in FIRST_k(Xv_i)$, and γ is the shortest prefix of $X\delta$ such that $\left.\gamma \Rightarrow^* a\omega\right\}$.
   (b) If $\left[X \to \alpha Y\bullet\beta, u, v, \gamma\right] \in D_j$, $Y \in N$ and $Y \to \delta \in P$, then $D_j := D_j \cup \left\{\left[Y \to \delta\bullet, u', v, \gamma\right]\right\}$, $u' = \mathrm{LAST}_q(\mathrm{BEFORE}_q(Y)\delta)$.
   (c) Repeat step (3b) till no new item can be added into D_j.
   (d) $C := C \cup \left\{D_j\right\}$.
4. Repeat step (3) for all created sets till no new set can be added into C.

Algorithm 9 Computation of $\mathrm{PSLS}(x, y)$ strings for $LLP(q, k)$ grammars.
Input A collection C of sets of $LLP(q, k)$ items for context-free grammar $G = (N, T, P, S)$, a pair of strings (x, y), $0 \le |x| \le q$, $1 \le |y| \le k$.
Output The value of $\mathrm{PSLS}(x, y)$.

1. Set $\mathrm{PSLS}(x, y) := \emptyset$.
2. Repeat for each $LLP(q, k)$ item:
   (a) Find a new $LLP(q, k)$ item of the form $\left[Y \to \alpha z\bullet\beta, x, y, \gamma\right]$, $x, y \in T^*$, $z \in T$, $\alpha, \beta, \gamma \in (N \cup T)^*$.
   (b) Set $\mathrm{PSLS}(x, y) := \mathrm{PSLS}(x, y) \cup \{\gamma\}$.

Algorithm 9 can be used to evaluate admissible pairs. A pair (x, y) is admissible in the grammar if and only if $|\mathrm{PSLS}(x, y)| > 0$.

Definition 10 *Let $G = (N, T, P, S)$ be a context-free grammar, AP be a set of admissible pairs of strings for language $L(G)$. Grammar G is an $LLP(q, k)$ grammar, if for each $(x, y) \in AP$, $|x| \le q$, $|y| \le k$, $|\mathrm{PSLS}(x, y)| = 1$.*

Example 11 Let us compute the collection C of sets of $LLP(1,1)$ items for grammar $G = (N, T, P, S)$, where P contains the following rules:

$$\begin{array}{llllllll}
(0) & S & \rightarrow & \vdash E \dashv & (1) & E & \rightarrow & TE' \\
(2) & E' & \rightarrow & +TE' & (3) & E' & \rightarrow & \varepsilon \\
(4) & T & \rightarrow & a & (5) & T & \rightarrow & [E]
\end{array}$$

The collection is listed below. The $LLP(1,1)$ items that define the contents of the *PSLS* table are marked by$\rightarrow (x, y)$, where x and y denote the corresponding table entry.

$$\begin{array}{llllllll}
\# & = \{ [& S & \rightarrow \vdash E \dashv \bullet, & \dashv, & \varepsilon, & \varepsilon &] \} & \\
\dashv & = \{ [& S & \rightarrow \vdash E \bullet \dashv, & a \mid], & \dashv, & \dashv &] & \\
 & [& E & \rightarrow TE' \bullet, & a \mid], & \dashv, & \dashv &] & \\
 & [& E' & \rightarrow +TE' \bullet, & a \mid], & \dashv, & \dashv &] & \\
 & [& E' & \rightarrow \bullet, & a \mid], & \dashv, & \dashv &] \} & \\
E_1 & = \{ [& S & \rightarrow \vdash \bullet E \dashv, & \vdash, & a \mid [, & E &] \} & \rightarrow (\vdash, a \mid [) \\
E'_1 & = \{ [& E & \rightarrow T \bullet E', & a \mid], & \dashv, & E' \dashv &] & \\
 & [& E & \rightarrow T \bullet E', & a \mid], & +, & E' &] & \\
 & [& E' & \rightarrow +T \bullet E', & a \mid], & \dashv, & E' \dashv &] & \\
 & [& E' & \rightarrow +T \bullet E', & a \mid], & +, & E' &] & \\
 & [& T & \rightarrow a \bullet, & a, & \dashv, & E' \dashv &] & \rightarrow (a, \dashv) \\
 & [& T & \rightarrow a \bullet, & a, & +, & E' &] & \rightarrow (a, +) \\
 & [& T & \rightarrow [E] \bullet, &], & \dashv, & E' \dashv &] & \rightarrow (], \dashv) \\
 & [& T & \rightarrow [E] \bullet, &], & +, & E' &] \} & \rightarrow (], +) \\
\vdash & = \{ [& S & \rightarrow \bullet \vdash E \dashv, & \varepsilon, & \vdash, & \vdash &] \} & \\
T & = \{ [& E & \rightarrow \bullet TE', & \vdash \mid [, & a \mid [, & T &] & \\
 & [& E' & \rightarrow +\bullet TE', & +, & a \mid [, & T &] \} & \rightarrow (+, a \mid [) \\
a & = \{ [& T & \rightarrow \bullet a, & \vdash \mid + \mid [, & a, & a &] \} & \\
] & = \{ [& T & \rightarrow [E \bullet], & a \mid], &], &] &] & \\
 & [& E & \rightarrow TE' \bullet, & a \mid], &], &] &] & \\
 & [& E' & \rightarrow +TE' \bullet, & a \mid], &], &] &] & \\
 & [& E' & \rightarrow \bullet, & a \mid], &], &] &] \} & \\
+ & = \{ [& E' & \rightarrow \bullet + TE', & a \mid], & +, & + &] \} & \\
E_2 & = \{ [& E & \rightarrow [\bullet E], & [, & a \mid [, & E &] \} & \rightarrow ([, a \mid [) \\
E'_2 & = \{ [& E & \rightarrow T \bullet E', & a \mid], &], & E'] &] & \\
 & [& E & \rightarrow T \bullet E', & a \mid], & +, & E' &] & \\
 & [& E' & \rightarrow +T \bullet E', & a \mid], &], & E'] &] & \\
 & [& E' & \rightarrow +T \bullet E', & a \mid], & +, & E' &] & \\
 & [& T & \rightarrow a \bullet, & a, &], & E'] &] & \rightarrow (a,]) \\
 & [& T & \rightarrow a \bullet, & a, & +, & E' &] & \rightarrow (a, +) \\
 & [& T & \rightarrow [E] \bullet, &], &], & E'] &] & \rightarrow (],]) \\
 & [& T & \rightarrow [E] \bullet, &], & +, & E' &] \} & \rightarrow (], +) \\
[& = \{ [& T & \rightarrow \bullet [E], & \vdash \mid + \mid [, & [, & [&] \} &
\end{array}$$

Example 12 Let us compute PSLS for all admissible pairs for grammar G from Example 11.

	a	$+$	$[$	$]$	$\dashv$
$\vdash$	E		E		
a		E'		$E']$	$E' \dashv$
$+$	T		T		
$[$	E		E		
$]$		E'		$E']$	$E' \dashv$

Note that the presented grammar is not an $LLP(0,1)$ grammar, since the admissible pair (ε, a) would require initial pushdown store contents either E or T. However, it is an $LLP(1,1)$ grammar.

Now we are ready to construct a deterministic parallel $LLP(q,k)$ parser. The processor network will be the same as that in the nondeterministic version. Since we added the left and right markers, we know that the last operation in parsing will be the comparison of the right marker. Therefore, we no longer need the additional processor that parsed the ε-suffix in the nondeterministic version.

When the parsing starts, each leaf processor establishes its lookahead and lookback strings. These strings are then used to index the PSLS table, where each processor obtains its initial pushdown store contents. The only exception is the first processor P_{11} that parses the left marker $\vdash$. This processor does not use the PSLS table to obtain the initial pushdown store contents. Instead, it knows that its initial pushdown store contents is the starting symbol of the grammar.

After this phase, each processor knows its initial pushdown store contents. Thus, the parsing of the assigned symbol can start as described in Sect. 3.1.1. The parsing provides intermediate results – the triplets of strings that are consequently combined into the final result as described in Sect. 3.1.2.

The parallel parser can be optimized in the parsing phase. If the grammar is $LLP(q,k)$ for some q and k, then the lookahead and lookback strings positively determine the initial pushdown store contents. Since the symbol that is to be parsed is in fact the first symbol of the lookahead string and LL parsing is deterministic, the lookahead and lookback string also positively determine the final pushdown store contents and the portion of the left parse. Therefore, we can precalculate a table that will hold all three information items (initial pushdown store contents, final pushdown store contents, and a portion of the left parse) and use this table instead of the PSLS table. If this table is employed, the parsing phase can be entirely replaced by indexing this precalculated table. We have named the table the LLP parsing table.

Algorithm 13 Computation of the LLP parsing table for grammar G
Input An $LLP(q,k)$ context-free grammar $G = (N, T, P, S)$, parsing table for G, PSLS table for G, and a pair of admissible input strings x, y, $|x| \leq q$, $|y| \leq k$.
Output The value of the LLP parsing table for the pair (x, y).

1. Use the PSLS table to obtain the initial contents of the pushdown store for the admissible pair (x, y). Let the initial contents be α and let a be the first symbol of y.
2. Perform sequential LL parsing from the initial configuration (α, a, ε). Stop after the comparison of a. Let the configuration just after the comparison be $(\varepsilon, \omega, \pi)$.
3. The value of the LLP parsing table for the pair (x, y) will be (α, ω, π).

Example 14 Let us compute the LLP parsing table for the grammar from Example 11.

	a	$+$	$[$	$]$	$\dashv$
$\vdash$	$(E, E', 14)$		$(E, E]E', 15)$		
a		$(E', TE', 2)$		$(E'], \varepsilon, 3)$	$(E' \dashv, \varepsilon, 3)$
$+$	$(T, \varepsilon, 4)$		$(T, E], 5)$		
$[$	$(E, E', 14)$		$(E, E]E', 15)$		
$]$		$(E', TE', 2)$		$(E'], \varepsilon, 3)$	$(E' \dashv, \varepsilon, 3)$

Example 15 Let us demonstrate deterministic parallel parsing based on the LLP parsing table for input string $\vdash a + [a + a] \dashv$. The parsing is depicted in Fig. 3.

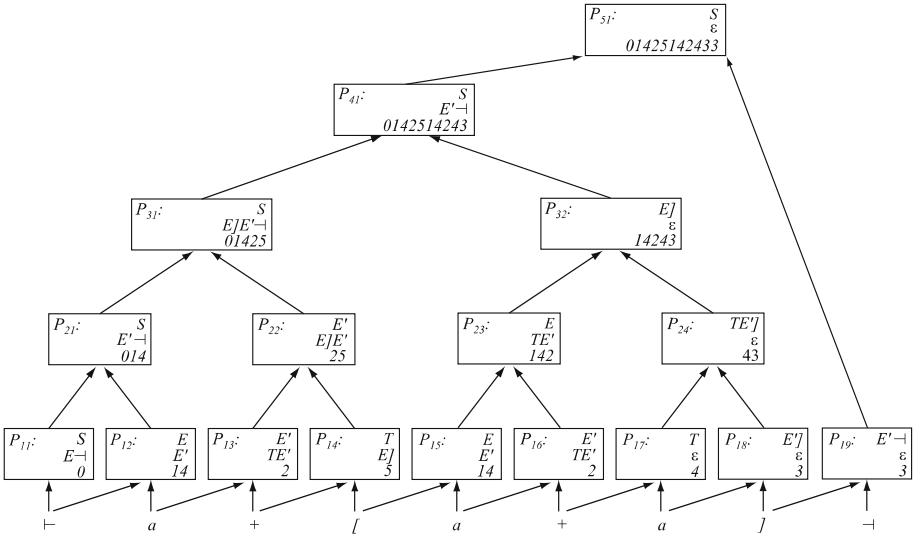

Fig. 3 Deterministic parallel LL parsing for input string $\vdash a + [a + a] \dashv$

3.5 LLP languages

The deterministic variant of the LLP parser can be constructed only for a subset of LL languages. The reason why the set of LLP languages is smaller than the set of LL languages lies in the fact that the parallel parser has only limited information on how to start the parsing. Compared to the sequential variant, the parallel version has lost information on the state of the characteristic automaton. This is clear from Algorithm 9. When filling in PSLS entries, only lookback and lookahead strings are considered. The characteristic automaton state (represented by the name of the set of LLP items) is not taken into account. Let us demonstrate some grammars that are LL grammars but are not LLP grammars for any fixed lengths of lookback and lookahead strings.

Example 16 Given the context-free grammar $G = (N, T, P, S)$ where the rules in P are:

$$\begin{array}{llllllll} (1) & S & \rightarrow & aA & (2) & S & \rightarrow & bB \\ (3) & A & \rightarrow & cAdA & (4) & A & \rightarrow & f \\ (5) & B & \rightarrow & eBdB & (6) & B & \rightarrow & f \end{array}$$

This grammar is $LL(1)$, however, it is not $LLP(q, k)$ for any fixed q and k. The reason is that the admissble pair (d, f) can be parsed with initial pushdown store contents equal to either A or B. To choose the correct one, the parser needs to know which rule was used for the expansion of S at the start of the parsing. However, this information is not within any bounded context, and the entire parsing history is needed here.

In the introducton, we stated that there exist even regular languages that cannot be parsed by the parallel LLP parser. Let us present one such example

Example 17 Let $L = \{a^{2n} : n \geq 0\}$. This language is regular, and it can be generated, for instance, by the following grammar:

$$(1) \quad S \rightarrow aaS \qquad (2) \quad S \rightarrow \varepsilon$$

The presented grammar is $LL(1)$, but is not $LLP(q,k)$ for any fixed q and k. Any grammar that generated this language must somehow pair the occurences of a's. Our grammar pairs even and odd a's. When trying to parse this language in parallel, the processor must distinguish whether the a that it is to parse is the first one or the second one in the pair. In our example grammar, the processor must know whether the a that it is to process is even or odd. However, the only information available is the lookback string, which is always of the form a^{*q}, and this does not help for input symbols in positions $q+1$, $q+2$, etc.

3.6 An optimal EREW PRAM algorithm

The advantage of parallel reduction is that the algorithm is very simple and can be used on any parallel architecture. Next, gluing described as parallel reduction is easier to understand, and is convenient for proofs. For these reasons, we have followed the model of parallel reduction up to now. Parallel reduction would imply logarithmic time complexity of the parallel LL parser. We will show that this is not true for some grammars and input strings.

Let us assume grammar G generating balanced bracket strings. The grammar may have the rules:

$$\begin{array}{ll} (0) & S' \rightarrow \vdash S \dashv \\ (1) & S \rightarrow [S] \\ (2) & S \rightarrow \varepsilon \end{array}$$

Such a grammar is an $LLP(1,1)$ grammar. Let the input string be $\vdash [^{2^l-1}]^{2^l-1} \dashv$. The processor in the root of the reduction tree will have to glue triplets:

$$(S', S]^{2^l-1} \dashv, 01^{2^l-1}) \quad \text{and} \quad (S]^{2^l-1} \dashv, \varepsilon, 2).$$

This gluing needs $2^l = 0.5n$ comparisons, i.e., one gluing operation would take up to $O(n)$ time and $T(n,n) = O(n \log(n))$ in this case. If we analyze this case in a greater detail, we obtain $T(n,n) = O(\sum_{i=0}^{\log(n)} \frac{n}{2^i}) = O(n)$. This is still a bad result. Althought n processors are employed, the parallel time is asymptotically linear as in the sequential algorithm.

This section describes a gluing replacement that achieves logarithmic time for any LLP grammar and any input string, thus it is time-optimal. The main problem of gluing is that a processor sometimes cannot do anything else but concatenate the contents of the initial (or final) pushdown store contents and pass this longer intermediate result to its parent. In such cases, the computational power of the child processor is wasted, as it indeed does nothing useful. The only way to ensure that the gluing phase is efficient is to split the problem evenly among all processors.

Recall the sequential algorithm that checks whether a string of brackets is balanced or not. The algorithm passes the input symbols from left to right. When it finds a left bracket, it pushes a symbol onto the pushdown store, and when it finds right bracket, it pops a symbol from the pushdown store (the symbol type may be further compared if more than one bracket type is used). The input string is balanced iff:

- the pushdown store is empty after the entire input has been read,
- no pop failed (the pushdown store was never empty when trying to pop a symbol),
- no mismatch in the symbol type test after the pop occurred.

Time-optimal gluing is based on a reverse process. We know the changes in the pushdown store contents (the triplets) and we need to know whether or not these changes leave the pushdown store empty, whether the corresponding symbol types match, and whether the changes do not require popping from an empty stack. Based on the above sequential algorithm, these three criteria are satisfied if a string of brackets corresponding to the pushdown store changes (that is, a string derived from the triplets) is balanced.

The triplet (α, ω, π) describes a change of the pushdown store contents. The change means that symbols in α are popped from the pushdown store. We will transform them into right brackets in the derived string. Similarly, symbols from the final pushdown store contents ω are to be pushed, thus they will be converted as left brackets. The sequence of triplets will provide us with a string that consists of brackets. We will check whether the string is balanced or not. If it is, the sequence of pushdown store contents changes is correct and the gluing succeeds. We will use the parallel bracket matching algorithm [19] for checking the string. The algorithm is suitable for PRAM only.

Let $G = (N, T, P, S')$ be an augmented context-free $LLP(q, k)$ grammar, $\beta = a_1 a_2 \cdots a_n$ be an input string and let the PRAM model have $p = n$ processors. After the parsing phase, each processor holds triplet τ_i of the form $(\alpha_i, \omega_i, \pi_i)$. Let us define two homomorphisms:

$$LBR(x) = \begin{cases} \varepsilon & x = \varepsilon \\ [^x & x \in (N \cup T) \\ [^y\, LBR(\gamma) & x = y\gamma, y \in (N \cup T), \gamma \in (N \cup T)^* \end{cases}$$

$$RBR(x) = \begin{cases} \varepsilon & x = \varepsilon \\ ]^x & x \in (N \cup T) \\ ]^y\, RBR(\gamma) & x = y\gamma, y \in (N \cup T), \gamma \in (N \cup T)^* \end{cases}$$

The key idea is that gluing of $\tau_1, \tau_2, \ldots, \tau_n$ produces the valid result triplet (S', ε, π) if and only if $\alpha_1 = S'$ and the string

$$LBR(\omega_1^R)RBR(\alpha_2)LBR(\omega_2^R)RBR(\alpha_3) \cdots LBR(\omega_{n-1}^R)RBR(\alpha_n)LBR(\omega_n^R)$$

forms a balanced bracket string of $(|N| + |T|)$-type brackets (in this notation the type of bracket is the upper index). Note that ω_n is essentially equal to ε, otherwise the string cannot be balanced.

Algorithm 18 Time optimal gluing algorithm for EREW PRAM.
Input Intermediate result triplets $\tau_1, \tau_2, \ldots, \tau_n$ from the parsing phase.
Output Signaling whether or not gluing leads to the accept triplet, left parse if it does.
Method For all processors do in parallel:

1. Evaluate local homomorphisms $RBR(\alpha_i)$ and $LBR(\omega_i^R)$, processor number 1 does not perform $RBR(\alpha_1)$.
2. Pack the homomorphism results (using string packing).
3. Evaluate matching bracket pairs. The parallel bracket matching algorithm is used in this step. Only left and right brackets are distinguished, bracket type (upper index) is not taken into account. This step evaluates an array of indices, and for any bracket in the string, the index of its mate is known.
4. Check that all matching bracket pairs are of the same type. Since indices of the corresponding bracket in the pair are known, this comparison can be done efficiently in parallel.
5. Perform string packing on the partial left parses π_i, producing left parse π.

The algorithm checks whether or not the input can be correctly glued in steps (3) and (4). Both these steps must succeed. If either fails, the input string is rejected. Clearly, step (5) is performed for valid input strings only.

Let us discuss the time complexity of the algorithm, step by step:

1. Since the length of both α_i and ω_i is limited by some constant z for the grammar, the time complexity is $O(z) = O(1)$.
2. The string packing step requires the parallel prefix sum ($O(\log(n))$) and copying of the strings ($O(z)$), thus this step takes time $O(\log(n))$.
3. Parallel Bracket Matching requires time $O(\log(n))$, see [19] for details.
4. Bracket type checking can be accomplished in $O(z) = O(1)$. This step, however, requires additional reduction and a broadcast informing all processors whether or not the bracket types matched everywhere in the string, thus the time will be $O(\log(n))$.
5. String packing is again $O(\log(n))$.

The above discussion shows that the overall time is $O(\log(n))$ if $p = n$ processors are employed, and time $O\left(\frac{n}{p} + \log(p)\right)$ if the number of processors is fewer than n. That is, the algorithm is not only time optimal, but also cost optimal if $p = \frac{n}{\log(n)}$. On the other hand, the hidden multiplicative constant may be very high here since the algo-

Input string: $\vdash a + [a + a] \dashv$
The triplets are:

	Triplet	$RBR(\alpha)$	$LBR(\omega^R)$
τ_1	$(S, E \dashv, 0)$	N/A	$[^{\dashv}[^{E}$
τ_2	$(E, E', 14)$	$]^{E}$	$[^{E'}$
τ_3	$(E', TE', 2)$	$]^{E'}$	$[^{E'}[^{T}$
τ_4	$(T, E], 5)$	$]^{T}$	$[^{]}[^{E}$
τ_5	$(E, E', 14)$	$]^{E}$	$[^{E'}$
τ_6	$(E', TE', 2)$	$]^{E'}$	$[^{E'}[^{T}$
τ_7	$(T, \varepsilon, 4)$	$]^{T}$	ε
τ_8	$(E'], \varepsilon, 3)$	$]^{E'}]^{]}$	ε
τ_9	$(E' \dashv, \varepsilon, 3)$	$]^{E'}]^{\dashv}$	ε

The string after homomorphism from step (1) and string packing from step (2) is applied:

$$\underbrace{[^{\dashv}_{1}[^{E}_{2}}_{\tau_1}\ \underbrace{]^{E}_{3}[^{E'}_{4}}_{\tau_2}\ \underbrace{]^{E'}_{5}[^{E'}_{6}[^{T}_{7}}_{\tau_3}\ \underbrace{]^{T}_{8}[^{[}_{9}[^{E}_{10}}_{\tau_4}\ \underbrace{]^{E}_{11}[^{E'}_{12}}_{\tau_5}\ \underbrace{]^{E'}_{13}[^{E'}_{14}[^{T}_{15}}_{\tau_6}\ \underbrace{]^{T}_{16}}_{\tau_7}\ \underbrace{]^{E'}_{17}]^{]}_{18}}_{\tau_8}\ \underbrace{]^{E'}_{19}]^{\dashv}_{20}}_{\tau_9}$$

The bracket matching algorithm evaluates pairs:

i	1	2	3	4	5	6	7	8	9	10
MATCH[i]	20	3	2	5	4	19	8	7	18	11
i	11	12	13	14	15	16	17	18	19	20
MATCH[i]	10	13	12	17	16	15	14	9	6	1

Step (4) compares bracket types. In our example all bracket types do match.
Step (5) produces left parse 0 14 2 5 14 2 4 3 3

Fig. 4 Deterministic parallel *LL* parsing with time optimal gluing

rithm requires several parallel prefix sums and reductions; the hidden multiplicative constant for parallel bracket matching algorithm is also high.

Example 19 Let us demonstrate the optimal gluing algorithm on the grammar from Example 11. The parsing of the input string $\vdash a + [a + a] \dashv$ is depicted in Fig. 4.

4 Conclusion

The parallel deterministic LL parser has been introduced and a class of LLP grammars suitable for deterministic parallel LL parsing has been defined. The LLP grammars form a proper subset of LL grammars. The subset includes important languages, such as the example arithmetic expression language, on the other hand, there are even regular languages which cannot be described by a $LLP(q, k)$ grammar for any fixed q and k. The presented parallel LL parser is very simple and fast, because the parsing can be done by indexing in the PSLS table and a parallel reduction.

References

1. Adriaens, G., Hahn, U., (eds.): Parallel Natural Language Processing. Ablex Publishing Corporation, Norwood (1994)
2. Aho, A.V., Ullman, J.D.: The theory of parsing, translation and compiling, parsing, vol. 1, compiling, vol. 2. Prentice-Hall Inc., Englewood Cliff (1972)
3. Andrei, S.: Bidirectional parsing. Ph.D. Thesis, Fachbereich Informatik, Universiteit Hamburg, Germany, 2000, pp. 39–54. Available in electronic form: http://www.sub.unihamburg.de/disse/134/inhalt.html
4. Aycock, J., Horspool, N., Janoušek, J., Melichar, B.: Even faster generalized *LR* parsing. Acta Inf. **37**, 633–651 (2001)
5. Bunt, H., Tomita, M.: Recent Advances in Parsing Technology. Kluwer Academic Press, Dordrecht (1996)
6. Cole, M.: List homomorphic parallel algorithms for bracket matching. Department of Computer Science, University of Edinburgh, CSR-29-93 (1993)
7. Chang, J.H., lbarra, O.H., Palis, M.A.: Parallel parsing on a one-way array of finite-state machines. IEEE Trans. Comput. **36**, 64–75 (1987)
8. Chiang, Y.T., Fu, K.S.: Parallel parsing algorithms and VLSI implementations for syntactic pattern recognition. IEEE Trans. Pattern Anal. Mach. Intell. **6**, 302–314 (1984)
9. Gibbons, A., Rytter, W.: Efficient Parallel Algorithms, Chapt. 4. Cambridge University Press, London (1988)
10. Hill, J.M.D.: Parallel lexical analysis and parsing on the AMT distributed array processor. In: Parallel Computing, pp. 699–714 (1992)
11. Ibarra, O.H., Pong, T.-C., Sohn, S.M.: Parallel recognition and parsing on the hypercube. IEEE Trans. Comput. **40**, 764–770 (1991)
12. Janoušek, J.: Some new results in sequential and parallel (generalized) LR parsing and translation. Ph.D. Thesis, CTU FEE Prague (2001)
13. Kosaraju, S.: Speed of recognition of context-free languages by array automata. SIAM J. Comput. **4**, 331–340 (1975)
14. Kurki-Suonio, R.: Notes on top–down languages. BIT **9**, 225–238 (1969)
15. Luttighuis, P.O.: Parallel Algorithms for Parsing and Attribute Evaluation. FEBO druk, Enschede, The Netherlands (1993)
16. Melichar, B., Vagner, L.: Parallel $LLP(q, k)$ parsing. In: Proceedings of Workshop 02, vol. A, CTU Prague, pp. 190–191 (2002)
17. Melichar, B., Vagner, L.: Time-optimal LLP parsing with parallel parentheses matching. In: POSTER 2002 – Book of Extended Abstracts, CTU Prague, Faculty of Electrical Engineering (2002)
18. Melichar, B., Vagner, L.: Formal parallel translation directed by parallel $LLP(q, k)$ parser. In: Proceedings of Workshop 03, CTU Prague, vol. A, pp. 236–237 (2003)

19. Prasad, S.K., Das, S.K., Chen, C.C.-Y.: Efficient EREW PRAM Algorithms for Parentheses-Matching. IEEE Trans. Parallel Distrib. Syst. **5**(9), 995–1008 (1994)
20. Ra, D.-Y., Kim, J.-H.: A parallel parsing algorithm for arbitrary context-free grammars. Inf. Process. Lett. **58**, 87–96 (1996)
21. Šaloun, P., Melichar, B.: Parallel Parsing of $LRP(q, k)$ Languages. MARQ, Ostrava (2002)
22. Shankar, P.: $O(log(n))$ parallel parsing of a subclass of $LL(1)$ languages. In: Parallel Computing, pp. 511–516 (1990)
23. Skillicorn, D.B., Barnard, D.T.: Parallel parsing on the connection machine. In: Information Processing Letters, pp. 111–117 (1989)
24. Tseytlin, G.E., Yushchenko, E.L.: Several aspects of theory of parametric models of languages and parallel syntactic analysis. In: Ershov, A., Koster, C.H.A. (eds.) Methods of Algorithmic Language Implementation, LNCS 47, pp. 231–245. Springer, Berlin Heidelberg New York (1977)